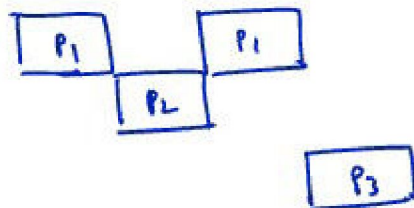


Virtualization

- CPU
- Memory (RAM)

Process

- Each process thinks that it has all the CPU
- multiple processes run (concurrently) at the same time.
- time slicing
 - runs part of one process & switches to other process - quickly



3 states:

- running
- blocked
- ready

I/O bound tasks

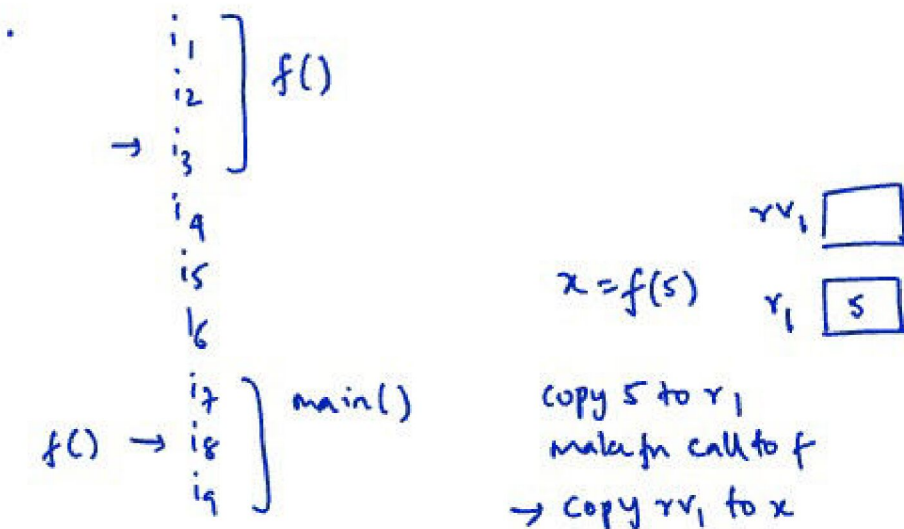
- orders of magnitude slower than running instructions

CPU bound tasks

- running normal instructions

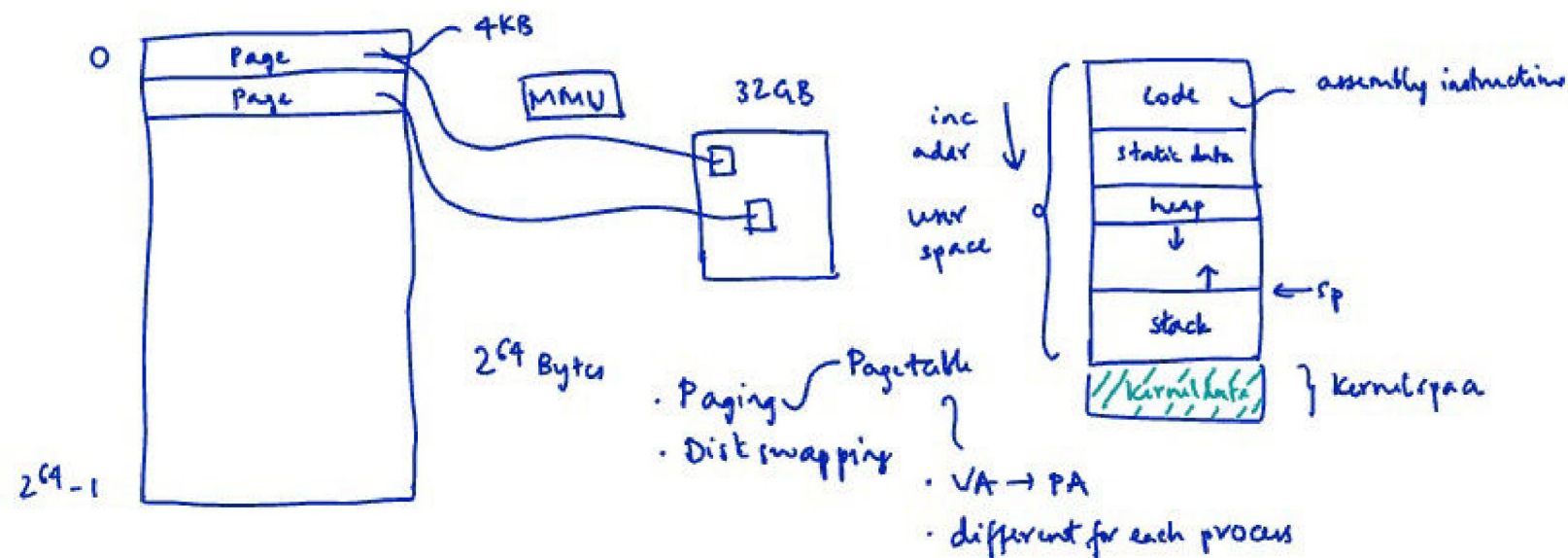
CPU

- Instruction pointer: stores the address of the next instruction to run
- Instructions are in memory
 - big array of bytes - addressable by bytes
- Registers
 - 32/64 registers
 - Some of them are assigned to store fixed things - stack ptr of current process, return address, frame ptr, etc
 - Some of them are general purpose
 - can be used for temporary calculations



Memory virtualization

- Each process has a virtual address space
- Each process thinks it has access to entire memory



- kernel mode
 - user mode
- we switch to kernel mode when OS wants to run something

Context switch

- state
 - registers
 - instruction ptr
 - page table address etc..
- store old process's content (state) in memory
- load the new process's content

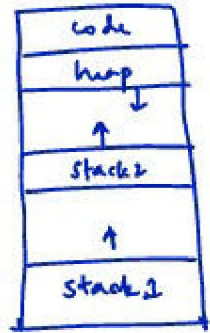
Inter process communication

- pipes
- named pipes - file
- FIFO queues etc

ls | wc
 } pipe
 takes output of ls
 & passes it as input to wc

- Threads

- different threads can belong to the same process
⇒ all of these share the virtual space
- each thread has its own stack



- advantages - why threads?
 - parallelization - if multiple CPUs
 - I/O bound tasks
- shared memory
 - context switch - lighter than a process

- race conditions / data races
- critical section
- mutual exclusion (mutex)

- Why not have an atomic super-instruction that does the mov-add-mov

- Synchronization primitives
 - locks
 - condition variables
 - semaphores etc

- goal: make concurrent programs correct & efficient using above primitives

- Issues with Concurrency

```

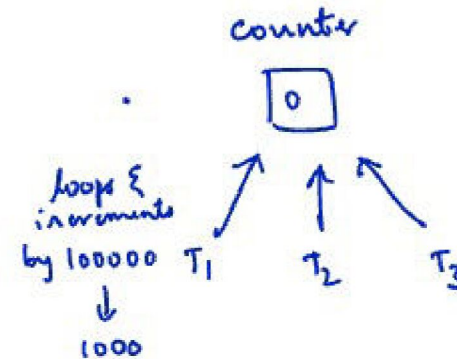
    .      x++      mov @maddr r5
              add    1, r5
              mov    r5, @maddr

```

 $x \begin{array}{|c} 50 \end{array}$

	T ₁	T ₂	addr
mov	50		50
add	51		50
		50	50
		51	50
		51	51
mov	51		(51)

- correct programs - should be correct all the time irrespective of the interleavings & scheduling counter



expected: 300000

actual: $\sim 200000 \text{ y}$

- wrong result
- a different result each time

locks

- acquire a lock before entering critical section

```
lock()
unlock()
trylock() }
timedlock() } T1 → // critical section code
unlock()
```

threads:

- create()
- destroy()
- join()

```
lock {
    queue of threads
    free or not
    thread ID of current lock owner
    ...
}
```

class NaiveCounter:

```
init():
    self.value = 0

inc(val=1):
    self.value += val

dec(val=1):
    self.value -= val
```

- perfect scaling

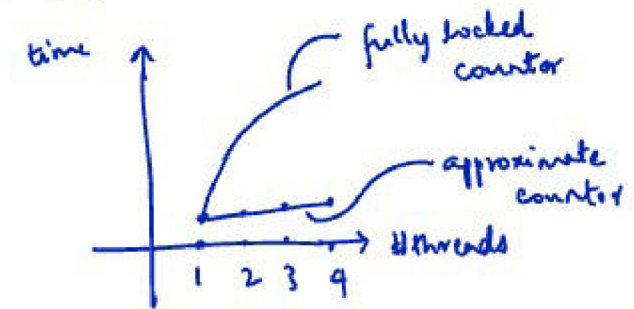
- 1 thread incrementing 10000 times - 1s
- 4 threads incrementing 10000 times each - 1s (assume 4 CPUs)

class LockCounter

```
init():
    self.value = 0
    self.lock = Lock()

inc(val=1):
    with lock:
        self.value += val
    ...
```

lock counter



• Approximate counter

- One local counter per CPU
- One global counter
- threshold S

• Idea:

- every time a local ctr crosses the threshold, we update the global counter

• Max error = $N \times S$
at any point
 ↓
 numcpus

- random CPU assignment } which CPU
thread-id % num_cpus } to use?

class ApproximateCounter:

init(threshold):

 global_counter = 0

 local_counters = [0 for _ in range(numcpus)]

 global_lock = Lock()

 local_locks = [Lock() for _ in range(numcpus)]
 self.threshold = threshold

inc(val=1):

 cpu_id = random_cpu_id()

 with local_lock[cpu_id]:

 local_counters[cpu_id] += 1

 if local_counters[cpu_id] ≥ self.threshold:

 with global_lock:

 global_counter += local_ctrs[cpu_id]

 local_ctrs[cpu_id] = 0

• Concurrent Linked List:

```
class LinkedList
```

```
init():
```

```
self.lock = Lock()
```

```
self.head = None
```

```
...
```

```
insert(val):
```

```
newnode = Node()
```

```
newnode.value = val
```

```
with self.lock:
```

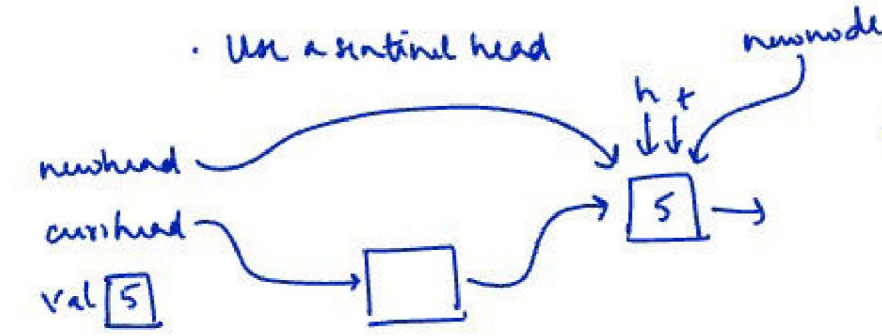
```
newnode.next = self.head
```

```
self.head = newnode
```

• Concurrent Queues:

- two locks - head lock & tail lock

- Use a sentinel head



dequeue

enqueue

create newnode

tail.next = newnode

currhead

newhead

update head

update tail

enqueue() - insertLast()

dequeue() - deleteFirst()

```
class ConcurrentQueue:
```

```
init():
```

```
head = tail = Node()
```

```
headlock = Lock()
```

```
taillock = Lock()
```

```
enqueue(val):
```

```
newnode = Node()
```

```
newnode.val = val
```

```
with taillock:
```

```
tail.next = newnode
```

```
tail = newnode
```

```
dequeue():
```

```
with headlock:
```

```
currhead = head
```

```
newhead = head.next
```

```
if not newhead: error
```

```
→ head = newhead
```

```
val = newhead.val
```

```
return val
```

	SLL (without tail)	SLL (with tail)	DLL
insertFirst()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
insertLast()	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
deleteFirst()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
deleteLast()	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$

Concurrent Hash tables (Sets Maps)

Buckets

Insert a key-value pair

$$i = \text{hash}(\text{key}) \% m$$



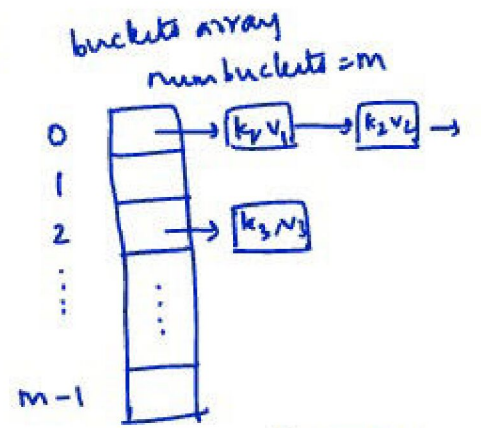
$[0, m-1]$

Collisions:

$$\text{key}_1 \neq \text{key}_2 \Rightarrow h(\text{key}_1) = h(\text{key}_2)$$

collision

$$h(\text{key}_1) \neq h(\text{key}_2) \Rightarrow \text{key}_1 \neq \text{key}_2$$



Separate chaining

$$\text{load factor} = \frac{n}{m} \leq 4$$

$$\text{Expected chain length} = n/m$$

class ConcurrentHashMap:

init():

buckets = array of concurrent linked lists

insert(key, val):

$$\text{id} = h(\text{key}) \% \text{len}(\text{buckets})$$

buckets[id].insert((key, val))

- scales much better than concurrent linked list because each bucket is locked/unlocked independently
- nearly perfect scaling

Condition Variables (CVs)

```

lock = lock()
done = False
cond = Cond()
child() {
    print('child')
    exit()
    return;
}
main() {
    print("parent")
    parent → create_child_thread();
    join()
    print("end");
}

1. lock()
done = True
cond.signal()
1. unlock()
parent
child
end

1. lock()
while not done:
    cond.wait(lock)
1. unlock()
  
```

Timeline

- parent calls wait()
 - ① CV releases the lock
 - ② & puts parent to sleep
 } atomically
 - parent - BLOCKED (on the CV)
- child calls signal() ⇒ parent - BLOCKED (on the CV) to BLOCKED (on the mutex lock)
- child calls unlock() ⇒ parent - BLOCKED (on the mutex lock) to READY
- parent is scheduled (lock has been reacquired) → wait() returns

Condition Variables

- we could wait on condition
- 2 functions:

- cond.wait(lock)
 - releases the lock
 - & then puts the caller thread to sleep → wait will only return once it reacquires the lock
- cond.signal()
 - wakes up any sleeping thread that is waiting on this cond

doesn't do anything with the lock

there could be more than one thread waiting on a particular CV

signal() - would wake one of them (which one is not specified)

if not atomic

CV releases the lock
→ child

acquire the lock

done = True

signals → no threads to wake
unlock

→ parent

put to sleep → nothing to wake it up
⇒ lost wake up

cond.broadcast() - wakes up all the threads that are waiting
notify_all()

- do we need the shared variable 'done'?

```
child() {
  l.lock();
  cond.signal();
  l.unlock();
}
```

- child runs first completely
- no threads woken up by signal
- parent waits forever

```
main() {
  l.lock();
  cond.wait(l);
  l.unlock();
}
```

- To solve the join problem - we could something like a spin wait/lock

```
child() {
  ...
  done = True;
}
```

- wastes CPU cycles
- sometimes it could be wrong

```
main() {
  ...
  while(!done);
  ...
}
```

spin lock / spin wait

- do we even need a lock?

```
child() {
  done = True;
  cond.signal();
}
```

we don't want other threads to sneak in between testing the shared data & waiting (lost wakeup issue)

```
main() {
  while not done:
    cond.wait()
}
```

- 3 rules with CVs

- have shared data along with the lock & CV.
- while over if
- call wait() / signal() with lock held

strict rule:
wait() needs the caller to hold the lock

some programs might work correctly in all cases even though lock isn't held but above problem (join) wouldn't work

• Bounded buffer / Producer - consumer problem

buffer = 0
count = 0 ✓
count == 0 ⇒ buffer is empty
count == 1 ⇒ buffer is full

```
put (int x) {  
    assert count == 0 ✓ throw an  
    buffer = x           Assertion error  
    count = 1  
}
```

```
get () {  
    assert count == 1  
    count = 0  
    return buffer  
}
```

target for a
producer
thread
(
producer():
 for i in range(10):
 put(i)

consumer():
 for i in range(10):
 print(get())

- assertion errors - producers trying put data into full buffer or viceversa
- data races - no locks around buffer, count

Cond has-items, has-space;

~~Cond~~, Lock L

producer() {

for i in range(10):

L.lock();

while if count != 0:

has-space.wait(L) ~~c.wait(L)~~

put(i)

~~c.signal()~~ has-items.signal()

L.unlock();

}

consumer() {

for i in range(10):

L.lock();

while if count == 0: has-items.wait(L)

~~c.wait(L)~~ →

print(get())

~~c.signal()~~ has-space.signal()

L.unlock();

}

- 1 producer + 1 consumer - code works fine no issues.
- 2 problems

• say 1 producer & 2 consumers

• T_{C1} blocked on CV

T_P acquires lock & put value & signals & blocks

T_{C1} is in Ready (not blocked on CV)

T_{C2} is scheduled & consumes the value & blocks

T_{C1} runs ⇒^{re}acquire lock & exit wait()

⇒ if ⇒ no check again ⇒ tries to get ⇒ assertion error as count is 0

⇒ we need to recheck state after waking up

⇒ while over if

• 2nd problem:

• 1 producer + 2 consumers

• T_{C1} runs & blocked

T_{C2} runs & blocked

T_P runs & puts value & signals & blocks

T_{C1} runs & gets value & signals & blocks

↓
which thread
would it wake?

Can't guarantee

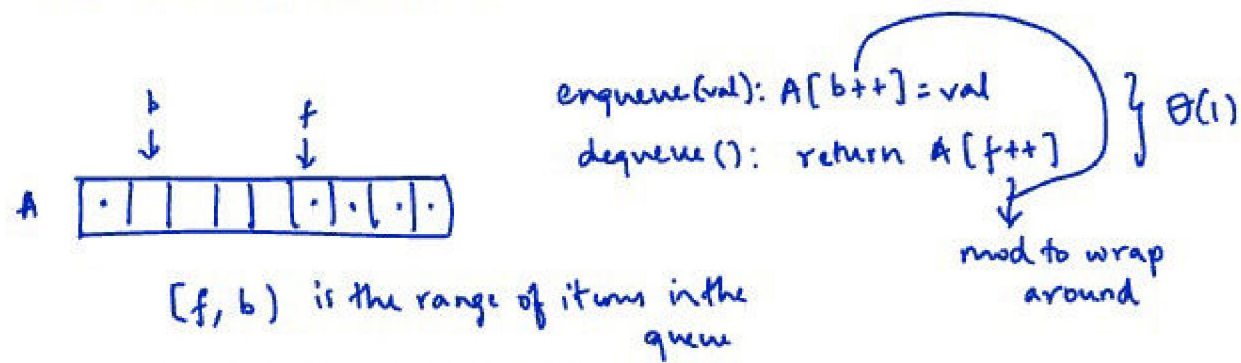
If T_{C2} gets woken up ⇒ blocks & all three
threads sleep forever

• Mesa semantics ⇒ the waking up is just a hint
↓
most implementations
use this strategy
to the thread that state has
changed
No guarantee that desired state
for the woken up thread

• Hoare semantics ⇒ guarantee that desired state
when woken up
- difficult to implement

Thread safe fns:

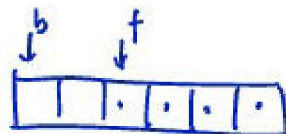
- A fn is thread-safe if even multiple threads call it concurrently, the result is always "correct" (not inconsistent)
- When making calls to external fns, make sure that they are thread safe, otherwise include that call in the critical section



$[f, b)$ is the range of items in the queue

$f == b \Rightarrow$ queue is empty
 or queue is full

eg: use a count variable (tracks #elements in the queue)
 or fill only $n-1$ elements in the n -len array



malloc (bytes)

memory full

- A malloc(100)
- B malloc(10)
- C free(50) $\rightarrow$ notify All() / broadcast()

Covering conditions

- consistently waking up all threads

• Semaphores

- Semaphore stores an integer value which we can use to initialize it

`s = Semaphore(3)`

- Once created, we generally can't access the value

`acquire()/wait()` would block if $\text{value} \leq 0$ before the call

- 2 methods:

- `sem_wait()` / `s.acquire()`

atomically

1. `s.value--`
2. wait if the value is now negative

- `sem_post()` / `s.release()`

↓
immediately returns

atomically

1. `s.value++`
2. If there are multiple threads waiting, it'll wake up one of them

- Semaphore as a lock

`s = Semaphore(1)`

`s.acquire()`

{ #critical section
....

`s.release()`

T_1
`acquire()`

runs critical section

`release()`

`s` [1]

T_2

`acquire()`

// blocked - has to wait
`val: -1`

T_3

`acquire()`

// blocked `val: -2`

critical section
`release()`

critical section
`release()`

- Invariant:

- when semaphore's $\text{value} < 0$
 $|\text{value}|$ represents #waiting threads

- Semaphores for ordering - join problem

```
child() {
    print(child)
    s.release()
    return;
}
```

```
main() {
    s = Semaphore(0)
    print(begin)
    create-child-thread()
    s.acquire() // join the child
    print(end)
}
```

expected
begin
child
end

s - 1
begin

- Useful lens:

- think of the semaphore's initial value as the # resources that are available to be immediately given away
- locks - 1 lock threads can acquire if initial value was 1
- join - no initial resources $\Rightarrow 0$ problem if child calls `release()` & creates the resource

$\rightarrow 0 \& 1$

```
child(id) {
    print("child" + id);
    s.release()
    return;
}
```

```
main() {
    print(begin)
    s = Semaphore(0)
    create-child-thread(child, 0)
    create-child-thread(child, 1)
    s.acquire()
    s.acquire()
    print(end)
}
```

expected output

begin		begin
child0	or	child1
child1		child0
end		end

- Bounded buffer problem with semaphores
- assume queue is NOT thread safe

```

prod-sem = Semaphore(N) → has_space
cons-sem = Semaphore(0) → has_items
lock = Semaphore(1)

```

```

producer(loops) {
    for i in range(loops):
        lock.acquire()
        has_space.acquire()
        q.enqueue(i)
        has_items.release()
        lock.release()
}

```

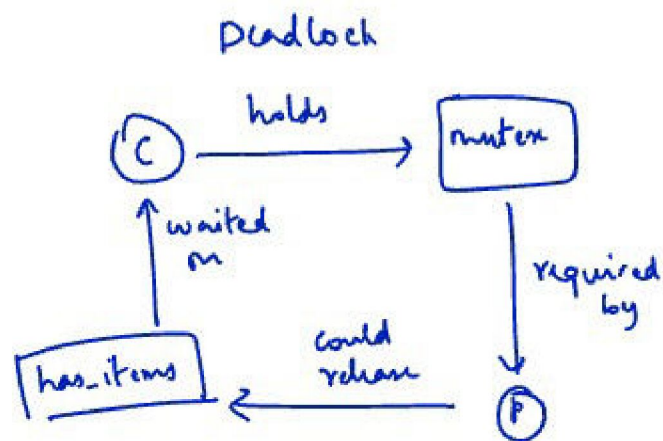
```

consumer(loops) {
    for i in range(loops):
        lock.acquire()
        has_items.acquire()
        val = q.dequeue()
        has_space.release()
        lock.release()
}

```

Consumer
acquires lock
blocked on has_items

Producer
blocked on lock



```

producer(loops) {
    for i in range(loops):
        has_space.acquire()
        [ q.enqueue(i)
          has_items.release() ]
}

consumer(loops) {
    for i in range(loops):
        has_items.acquire()
        [ val = q.dequeue()
          has_space.release() ]
}

```

lock this critical section

- or even better, make the queue thread-safe & no locks inside

• In CV implementation

1. CV releases the lock & reacquires it
⇒ no deadlock
2. Lock was doing things:
 1. locking the critical sections of interactions with queue
 2. interacting with the CV for us to be able to call wait() & signal() while holding the lock
3. Semaphore doesn't update the locks (interact)
 - we needed a separate for queue interactions in the queue to be thread safe
4. Unlike the CV implementation (which was relying on q.is_full() & q.is_empty() to not enqueue() in a full queue & to not dequeue() in an empty queue), semaphore implementation doesn't need them
⇒ We could remove 'count' from the queue implementation
⇒ Now we could use two-lock concurrent queue along with the semaphore implementation

• Reader-writer locks

- We want to allow multiple reader threads inside the critical section (but no writer thread)
- We want to only allow 1 writer thread (& no other reader threads) in the CS

- T₁ acquire-read-lock()
- T₂ acquire-read-lock()
- T₃ acquire-write-lock()
• blocked → ready
- T₂ release-read-lock()
- T₁ release-read-lock()
- T₃ runs
- T₄ acquire-read-lock()
• blocked in write lock
- T₅ acquire-read-lock()
• blocked in lock

lock	☒
write lock	☐ -1
num readers	☐ 1

T₁ - read ✓
 T₂ - read ✓
 T₃ - write X
 T₄ - read ✓
 ;

- starving all the writer threads
 - program doesn't stop from new readers to acquire a read lock even though there are writers waiting
- Reader preference RW lock

Writer preference - RW lock

- once a writer thread tries to acquire the write lock, we shouldn't let any new readers get the read lock

Thread throttling:

- a lot of threads running a particular function
- a certain section of this function is very memory intensive
- thrashing would happen if we allow all the threads to enter this section (too many page swaps from the disk)

s = Semaphore(k)

s.acquire()

< memory intensive region > — only want to allow k threads at any point

s.release()

Concurrency bugs

1. Non-deadlock bugs
2. Deadlock bugs

Atomicity violation $\Rightarrow$ use a lock
Order violation $\Rightarrow$ generally fixed using CVs / Semaphores

mThread = NULL

```
1 Thread 1::
2 void init() {
3     mThread = PR_CreateThread(mMain, ...);
4 }
5
6 Thread 2::
7 void mMain(...) {
8     mState = mThread->State;
9 }
```

global variable
running a new thread with mMain as target
accessing an instance variable State in mThread object

lock = Lock()

cond = Condition(lock)

init() {

with lock:

mThread = create_thread(mMain)

cond.signal()

}

mMain() {

with lock:

while mThread == NULL:

cond.wait()

mState = mThread.state

}

Deadlock

Thread 1

→ l₁.lock()
→ l₂.lock()
...

Thread 2

l₂.lock()
l₁.lock()
...

For a deadlock to occur, 4 conditions need to be met (ALL)

Mutual exclusion

Hold & wait

a thread while waiting for new resources doesn't release the resources it already acquired

No preemption

we cannot forcibly remove resources from thread

Circular wait

order of lock acquisition
fix this

eg: use ascending order of lock ids

f(l₁, l₂) if (l₁ > l₂) lock(l₂) lock(l₁)
else lock(l₁) lock(l₂)

l₁ = [1, 2, 3], l₂ = [4, 5]

l₁.extend(l₂) l₂.extend(l₁)

we could also have a partial ordering for all the locks globally

f(l₁, l₂)
lock(l₁)
lock(l₂)
...

relations

< for numbers

Total ordering:

every element is comparable with every other element

Fig: < on numbers

Partial ordering:

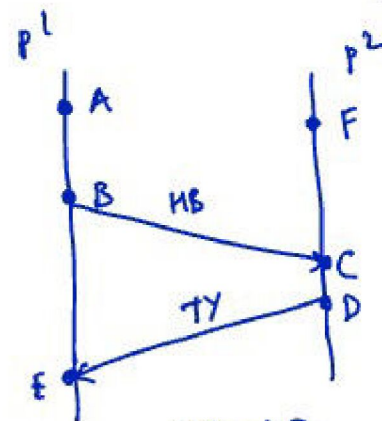
not a total ordering

eg. distributed systems

causal relationship

if A causes B

A → B



{ A → B
B → C
C → D
D → E

A, F not comparable

A, D

A → D

A, D comparable

- Hold & wait

```

lock.acquire()
l1.acquire()
l2.acquire()
:
lock.release()

```

we use a lock
for all the lock
acquisitions

- downsides:

- larger critical sections
(reduces concurrency)
- encapsulation

- No preemption:

- trylock()

top:

```

l1.lock()
l2.lock()
if l3.trylock() is not successful:
    l1.release()
    l2.release()
    goto top

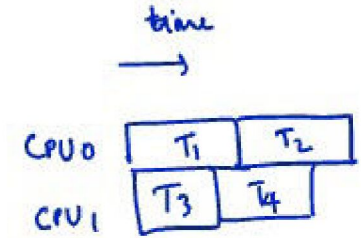
```

- Mutual exclusion:

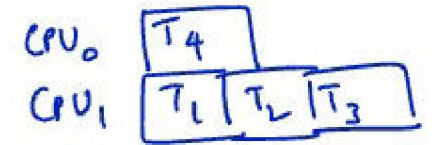
- wait-free & lock-free data structures
- not simple - sophisticated

- Deadlock avoidance through scheduling

	T ₁	T ₂	T ₃	T ₄
L ₁	✓	✓	X	X
L ₂	✓	✓	✓	X



	T ₁	T ₂	T ₃	T ₄
L ₁	✓	✓	✓	X
L ₂	✓	✓	✓	X



- DB: detect & recover

- DBMS runs these checks
periodically & builds the resource
graph
cycle ⇒ deadlock